Web 2 – Séance 03

Authentification par JWT

Authentification vs Autorisation

Authentification : Qui es-tu ? (vérifier l'identité)

- L'utilisateur se connecte avec email + mot de passe
- On valide ses identifiants
- On crée une session / token

Autorisation : Que peux-tu faire ? (vérifier les permissions)

- Peut-il accéder à cette recette ?
- Peut-il modifier cette recette ?
- Est-il administrateur ?

Authentification par JWT

- **JWT** = JSON Web Token
- Token = petit bout de texte encodé qui contient des informations sur l'utilisateur
- Représente l'identité de l'utilisateur et ses permissions
- Permet d'identifier l'utilisateur qui fait une action
- L'utilisateur est authentifié tant qu'il possède un token valide
- Le token est envoyé à chaque requête pour prouver l'identité de l'utilisateur

Le « fake token » du boilerplate vs JWT

Le boilerplate (comme Web 1) génère un token maison :

```
export const generateFakeToken = (email: string) ⇒ Buffer.from(email, "utf-8").toString("base64");
export const validateFakeToken = (token: string) ⇒ Buffer.from(token, "base64").toString("utf-8");
```

	Fake token	JWT
Contenu	uniquement l'email	ce qu'on veut inclure
Falsifiable	**oui** : il suffit d'encoder un email en Base64	**non** : signé avec une clé secrète connue du serveur seul
Expiration	jamais	`exp` intégré au token

→ Même principe (un texte envoyé à chaque requête), mais le JWT est **vérifiable** et **autoporteur**.

Structure d’un JWT

- Un token est un texte structuré avec 3 parties : `header.payload.signature`

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6MSwiZW1haWwiOiJqb2huQGdtYWlsLmNvbSIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNjM3NTEyMDAwLCJleHAiOjE2Mzc1OTg0MDB9.
79-zRXtICRfQFrMPWKb2GEo6A7DVEj-udz-g34KUp9Q
```

- **Header** : type de token et algorithme de signature
- **Payload** : données utilisateur (id, email, etc.)
- **Signature** : preuve d’authenticité

Chaque partie est encodée en Base64URL (variante de Base64 sans `+`, `/` ni `=`)
→ N’importe qui peut décoder le token et voir le contenu
https://jwt.io ou https://www.base64decode.org

Workflow JWT

Librairie `jsonwebtoken`

- Librairie à utiliser : `jsonwebtoken`
- Installation :

```
npm install jsonwebtoken
npm install --save-dev @types/jsonwebtoken
```

- Importation :

```
import jwt from "jsonwebtoken";
```

Configuration du JWT : Modèles nécessaires

```ts
// Informations identifiant l'utilisateur
// Stockées dans le payload du token
interface TokenPayload {
  id: number;
  email: string;
  role: "user" | "admin";
}

// Request étendue avec l'utilisateur authentifié (payload du token)
export interface AuthRequest extends Request {
  user?: TokenPayload;
}
```

Configuration du JWT : Clé secrète

La clé secrète ne doit **jamais** être écrite dans le code (ni dans Git)
Quiconque la connaît peut fabriquer des tokens valides.

Le boilerplate lance déjà le serveur avec `tsx --env-file=env/dev.env` : le fichier `env/dev.env` contient les variables d'environnement.

```
# env/dev.env
PORT=3000
JWT_SECRET=une_longue_chaine_aleatoire_de_32_caracteres_minimum
```

```
// Lecture dans le code : process.env.NOM_DE_LA_VARIABLE (string | undefined)
const SECRET_KEY = process.env.JWT_SECRET!;
```

- Chaque environnement (dev, test, prod) a son propre fichier et sa propre clé
- Les fichiers `.env` sont ajoutés au `.gitignore` pour ne pas être poussés sur GitHub

Générer un JWT

```
function generateToken(user: TokenPayload): string {
  return jwt.sign(
    user,
    SECRET_KEY,
    {
      expiresIn: "1d", // Expire dans 1 jour
      algorithm: "HS256", // algorithme de signature
    }
  );
}
```

Vérifier un JWT

```
function verifyToken(token: string): TokenPayload | undefined {
  try {
    const decoded = jwt.verify(token, SECRET_KEY) as TokenPayload;
    return decoded;
  } catch (error) {
    // Token invalide, expiré, etc.
    console.error("Token invalide :", error);
    return undefined;
  }
}
```

Middleware d'Authentification

Vérifier le token avant d'accéder à une route protégée.

```
export class AuthService {
  static authorize(req: AuthRequest, res: Response, next: NextFunction) {
    const token = req.get("Authorization");
    if (!token) return res.sendStatus(401);

    const payload = verifyToken(token);
    if (!payload) return res.sendStatus(401);

    req.user = payload; // disponible dans les middlewares et routes suivants
    next();
  }

  static isAdmin(req: AuthRequest, res: Response, next: NextFunction) {
    if (!req.user) return res.sendStatus(401);
    if (req.user.role !== "admin") return res.sendStatus(403);
    next();
  }
}
```

Routes Protégées

Utiliser le middleware pour protéger les routes.

```
recipesController.put("/:id", AuthService.authorize, (req: AuthRequest, res: Response) => {
  const recipeId = parseInt(req.params.id);
  const recipe = RecipesService.getRecipeById(recipeId);
  if (!recipe) return res.sendStatus(404);

  // Vérifier si l'utilisateur a le droit de modifier la recette : auteur ou admin
  // (req.user est forcément défini ici : le middleware authorize a répondu 401 sinon)
  if (recipe.authorId !== req.user!.id && req.user!.role !== "admin") {
    return res.sendStatus(403); // Forbidden
  }

  RecipesService.updateRecipe(recipeId, req.body);
  res.sendStatus(204);
});

recipesController.delete("/:id", AuthService.authorize, AuthService.isAdmin, (req: Request, res: Response) => {
  const recipeId = parseInt(req.params.id);
  if (!RecipesService.getRecipeById(recipeId)) return res.sendStatus(404);

  RecipesService.deleteRecipe(recipeId);
  res.sendStatus(204);
});
```

Route de Login

Vérifie les identifiants et génère un JWT si correct.

```
authController.post("/login", (req: Request, res: Response) ⇒ {
  const body: unknown = req.body;
  if (!isCredentialsDTO(body)) return res.sendStatus(400);

  const { email, password } = body;
  if (!AuthService.isValidCredentials(email, password)) return res.sendStatus(401);

  const token = generateToken({
    id: user.id,
    email: user.email,
    role: user.role,
  });

  res.json({ token });
});
```

Stockage d’un JWT dans REST Client

Permet de tester les routes protégées avec un token.

```
### Login
# @name login
POST http://localhost:3000/auth/login
Content-Type: application/json

{
  "email": "...",
  "password": "..."
}

### DELETE Recipe
DELETE http://localhost:3000/recipes/5
Authorization: {{login.response.body.token}}
```

Récapitulatif Séance 03

- **Authentification vs Autorisation** – Qui es-tu ? Que peux-tu faire ?
- Qu'est-ce qu'un **JWT** ? – JSON Web Token
- **JWT Structure** – header.payload.signature
- **Générer un JWT** – `jwt.sign()` avec clé secrète
- **Vérifier un JWT** – `jwt.verify()` pour décoder
- **Middleware** – Protéger les routes
- **Route de login** – Vérifier les identifiants et générer un token

Prochaine séance : Séance 04 – Hachage et sécurité

Exercice filé S03

1. Modifiez votre backend pour utiliser JWT pour l'authentification
2. Assurez-vous que votre route de login renvoie un token JWT
3. Protégez les routes qui vous semblent nécessaires pour qu'elles nécessitent un token JWT valide
4. Testez vos routes avec REST Client
5. **Optionnel** : Décodez un JWT manuellement sur jwt.io et retrouvez ses claims